

Reading 6.5: Actor Critic and PPO

Are you ready to discover how Proximal Policy Optimisations works, the algorithm used to align GPT4 to human values?

Welcome to this session on Actor-Critic methods, a cornerstone of modern Reinforcement Learning. After reading this document you will understand:

- The key components of Actor-Critic algorithms
- Trust Region Policy Gradients
- Proximal Policy Optimisation or PPO

Let's start from our policy gradient algorithm.

$$\hat{g} = \frac{1}{m} \sum_{i=1}^m \sum_{t=0}^{H-1} \nabla_{\theta} \log \pi_{\theta}(u_t^{(i)} | s_t^{(i)}) \left[\sum_{k=t}^{H-1} R(s_k^{(i)}, u_k^{(i)}) - b(s_t^{(i)}) \right]$$

Ideally, we want $\left(\hat{Q}_i(s_t^{(k)}, u_t^{(k)}) - V_{\phi_i}^{\pi}(s_t^{(k)}) \right)$

$$A_t = \left(\hat{Q}_i(s_t^{(k)}, u_t^{(k)}) - V_{\phi_i}^{\pi}(s_t^{(k)}) \right)$$

Advantage Function A_t

- Algorithm is called AC2
- Goal is to reduce variance

A good choice for b is the state value $V_{\pi}(s_t)$ and as we have collected rewards up to time t , we can calculate $V_{\pi}(s_t)$. $V_{\pi}(s_t)$ tells us how much reward we got based on the current policy. If the action that we take results in a better/worse reward, we will increase/decrease probability of that action going forward. We need to estimate $V_{\pi}(s_t)$ using a (different) neural network parametrised by ϕ , relying on Monte Carlo or TD.

As far as $\sum_{k=t}^{H-1} R(s_k^{(i)}, u_k^{(i)})$ goes, we want to determine the Q-value for that state-action pair

$$Q_{\pi}(s, u) = E[r_0, r_1, \dots]$$

We can decide how many rewards we want to sum up and this is a hyperparameter k .

$$Q_{\pi}(s, u) = E[r_0 + \gamma V_{\pi}(s_1)] \quad k=1$$

$$Q_{\pi}(s, u) = E[r_0 + \gamma r_1 + \gamma^2 V_{\pi}(s_2)] \quad k=2$$

$$Q_{\pi}(s, u) = E[\dots]$$

Algorithms where we rely on the value function $V_\phi(s)$ to represent the baseline b , fall under the umbrella of Actor-Critic algorithms. In Actor-Critic algorithms we have two components at play:

1. Actor: policy π_θ : component responsible for selecting actions
2. Critic: value function $V_\phi(s)$ component responsible for evaluating actions – feeds back how good the action taken was

Both the A2C (Advantage Actor-Critic) algorithm and the A3C (Asynchronous Advantage Actor-Critic) rely on the advantage A_t

$$A_t = \left(\hat{Q}_i(s_t^{(k)}, u_t^{(k)}) - V_{\phi_i}^\pi(s_t^{(k)}) \right)$$

The difference between A2C and A3C (2016) lies in how they handle synchronisation between workers during training. A2C is the foundation, focused on reducing variance in policy gradient updates using advantage functions, but it processes data sequentially and lacks parallelism. A3C builds on A2C by introducing asynchronous updates with multiple agents, making it faster, more scalable, and better suited for exploring large environments.

Natural Policy Gradients (TRPO)

Let's look at the 3rd option to reduce the variance of the policy gradient estimates: the KL Divergence Trust Region. Policy gradient has 3 issues:

1. Step Size Control: In RL, choosing the wrong step size can lead to a bad policy, generating poor data for the next update, causing non-recoverable performance collapse.
2. Non-Stationary Inputs: As the policy evolves, the distribution of observations and rewards changes, making training unstable
3. Sample Efficiency: RL performs only one gradient step per environment sample, which limits data efficiency.

Modern machine learning often reduces problems to numerical optimisation. In Policy Gradient we use stochastic gradients without directly defining a clear optimisation problem.

Our goal is to create an optimisation problem for RL that enables small, stable policy updates using on-policy data sampled from π . We need a loss function to guide policy gradient methods while ensuring that updates use accurate gradients.

TRPO

Let's start from the policy gradient formula, where the gradient of the expected reward is computed to iteratively improve the policy π_θ .

$$\hat{g} = E_t[\nabla_\theta \log \pi_\theta(u_t, s_t) A_t]$$

$\hat{g}$ is hard to optimise because A_t is very noisy

In Trust Region Policy Optimisation (TRPO) we approximate the complex non-linear function $\hat{g}$ by a simpler function, and optimise the simpler function instead. The region where the approximation of the simpler function is close to $\hat{g}$ is called the Trust Region. Mathematically this comes down to solving a constrained optimisation problem. The constraint tells us that distributions must be similar.

The formula for $\hat{g}$ can also be derived via a technique called importance sampling. In importance sampling you compute an expectation under some policy π_θ and you sample trajectories from another policy $\pi_{\theta_{old}}$. We define the surrogate loss as:

$$\text{Surrogate loss: } \max_{\pi} L(\pi) = \mathbb{E}_{\pi_{old}} \left[\frac{\pi(a|s)}{\pi_{old}(a|s)} A^{\pi_{old}}(s, a) \right]$$

- $\pi(a|s)$: This is the new policy (your new way of making decisions). It tells you the probability of taking action a in state s .
- $\pi_{old}(a|s)$: This is the old policy (the previous way of making decisions).
- $\frac{\pi(a|s)}{\pi_{old}(a|s)}$: This ratio measures how much the new policy changes compared to the old one. For example, if the new policy is twice as likely to take an action, the ratio will be 2.
- $A^{\pi_{old}}(s, a)$: This is the advantage function. It tells you how much better or worse a specific action a is compared to what you would normally do in that state s . If $A > 0$, it means this action is good; if $A < 0$, it's bad. 

The surrogate loss represents an objective designed to maximise the expected cumulative reward. It is not a "loss" in the traditional sense but rather a measure of policy improvement.

We add a constraint, to keep the policy updates safe and stable.

$$\mathbb{E}_{\pi_{\text{old}}} [KL(\pi || \pi_{\text{old}})] \leq \epsilon$$

This equation is about keeping the policy updates **safe and stable**. Here's what each term means:

- $KL(\pi || \pi_{\text{old}})$: This is called the **KL divergence**, which measures how different the new policy π is from the old policy π_{old} . If this value is small, the new policy hasn't changed much.
- $\mathbb{E}_{\pi_{\text{old}}} [\cdot]$: This means we take the average over all states and actions based on the old policy.
- $\leq \epsilon$: This ensures the change between π and π_{old} stays small and within a safe threshold ϵ .

In TRPO we must solve a constrained optimisation problem, which can be complex. Another issue is that the surrogate loss still relies on the noisy advantage A_t .

PPO

One of the alternatives to TRPO is Proxy Policy Optimisation or PPO. PPO simplifies the implementation while maintaining stability and performance. In PPO we replace the hard KL constraint with a clipping mechanism.

Clipped Surrogate Objective:

- PPO replaces the hard KL constraint with a **clipping mechanism** in the objective function:

$$L^{\text{CLIP}}(\theta) = \mathbb{E} [\min(r_t(\theta)A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)A_t)]$$

- $r_t(\theta)$: Probability ratio of new policy to old policy.
- The clip limits the probability ratio to the range $[1 - \epsilon, 1 + \epsilon]$, ensuring the new policy doesn't deviate too much from the old one.

PPO maintains the stability of TRPO and introduces a clipping mechanism to enforce trust regions implicitly rather than solving a hard-constrained optimisation problem. This makes PPO computationally efficient and widely used in practice.

Try it Yourself

Are you ready to take your understanding of PPO and Actor-Critic to the next level? Start by implementing PPO on a small environment and experiment with the clipping mechanism to see how it stabilises training in action!
